

Student Grade Prediction (ML Project)

Agenda

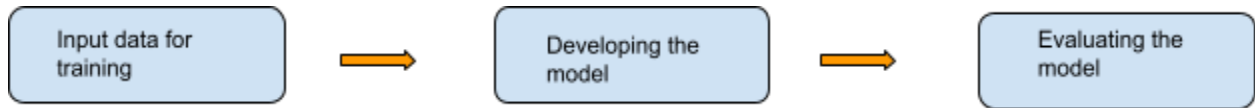
1. Why do we need Machine Learning
2. What is Machine Learning
3. Lifecycle to build a Model with ML
4. Supervised Learning: Linear Regression Vs Polynomial Regression
5. Student grade Prediction Using ML

1

- Earlier, most data was organized in structured formats. However, with the rapid growth of data today, structured data has become limited. To manage and analyze the massive amount of unstructured data, data science and machine learning techniques are essential.
- This data can uncover valuable business insights and hidden patterns that were previously unnoticed.
- These insights enable organizations to make better predictions about future outcomes.
- Machine learning also helps in reducing production costs by improving efficiency.
- By building models based on data, machines gain the ability to make predictions automatically without constant human input.

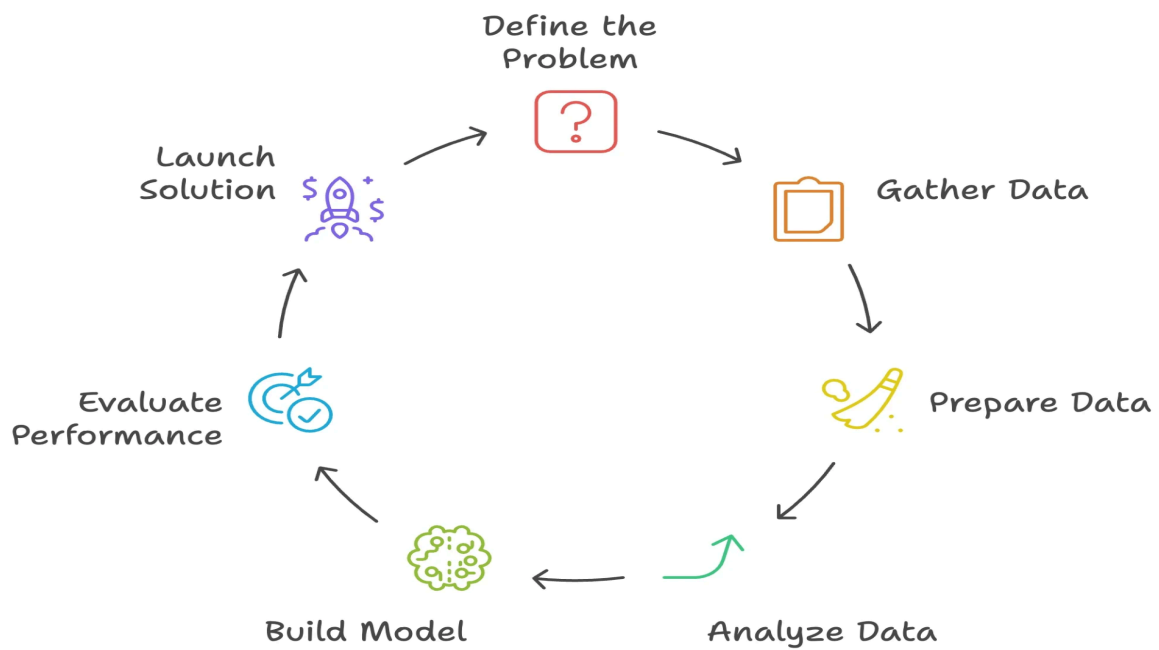
2

Machine learning is a field within artificial intelligence (AI) that enables systems to independently learn from data and enhance their performance over time, without requiring specific programming instructions.



3

Roadmap to creating a machine learning model



4

Linear Regression is a method used in machine learning to predict a value (called the target or dependent variable) using one or more independent variables (the inputs). It helps us understand how changes in one variable affect another.

- A method to predict a target value (Y) using one or more input variables (X).
- Helps find the relationship between inputs and output.
- It's a supervised machine learning algorithm (learns from labeled data).
- Works only with continuous values (e.g., marks, prices, temperature).
- Formula: $y = mx + c$
- $m \rightarrow$ slope of the line
- $c \rightarrow$ intercept (where line starts on Y-axis)
- Goal: Find the best-fitting line that gives the most accurate predictions.

Polynomial Regression is a machine learning method used when the relationship between input (X) and target (Y) is not a straight line but a curve.

- Predicts continuous values like scores, prices or heights.
- Still a supervised learning algorithm (uses Labeled data)
- Add power terms like X^2 , X^3 ... to capture bending patterns in data.
- Useful when data does not fit a straight line trend
- Formula: Degree 2 $Y = a + bX + cX^2$
- X^2 allows the line to bend
- More degrees $\rightarrow$ more flexibility to fit the pattern
- Goal: Find a smooth curved line that best matches the data points and improves prediction accuracy compared to simple linear regression.

```
[8]
✓ 20s
import pandas as pd
import numpy as np
from google.colab import files
uploaded = files.upload()

Choose files StudentPerf...Factors.csv
StudentPerformanceFactors.csv(text/csv) - 641949 bytes, last modified: 25/10/2025 - 100% done
Saving StudentPerformanceFactors.csv to StudentPerformanceFactors.csv

[9]
✓ 0s
data=pd.read_csv("/content/StudentPerformanceFactors.csv")

[ ]
data.head()
```

	Hours_Studied	Attendance	Parental_Involvement	Access_to_Resources	Extracurricular_Activities	Sleep_Hours	Previous_Scores	Motivation_Level	Internet_A
0	23	84	Low	High	No	7	73	Low	
1	19	64	Low	Medium	No	8	59	Low	
2	24	98	Medium	Medium	Yes	7	91	Medium	
3	29	89	Low	Medium	Yes	8	98	Medium	
4	19	92	Medium	Medium	Yes	6	65	Medium	

Final Grade

I uploaded the file `StudentPerformanceFactors.csv` into Google Colab and loaded it into a pandas DataFrame. The `data.head()` function shows the first rows of the dataset. This confirms that the file was imported correctly and gives an initial view of the features available for analysis.

This dataset includes factors such as hours studied, attendance, parental involvement, motivation, previous scores, and access to learning resources. These features will be used as predictors. The goal is to estimate the student's final exam score based on these inputs.

```
Final Grade

[10] ✓ Os data['Exam_Score'].describe()

Exam_Score
count 6607.000000
mean 67.235659
std 3.890456
min 55.000000
25% 65.000000
50% 67.000000
75% 69.000000
max 101.000000

dtype: float64
```

The `describe()` function is used to generate statistical insights for the target variable `Exam_Score`. This provides an overview of the central tendency and spread of the scores in the dataset.

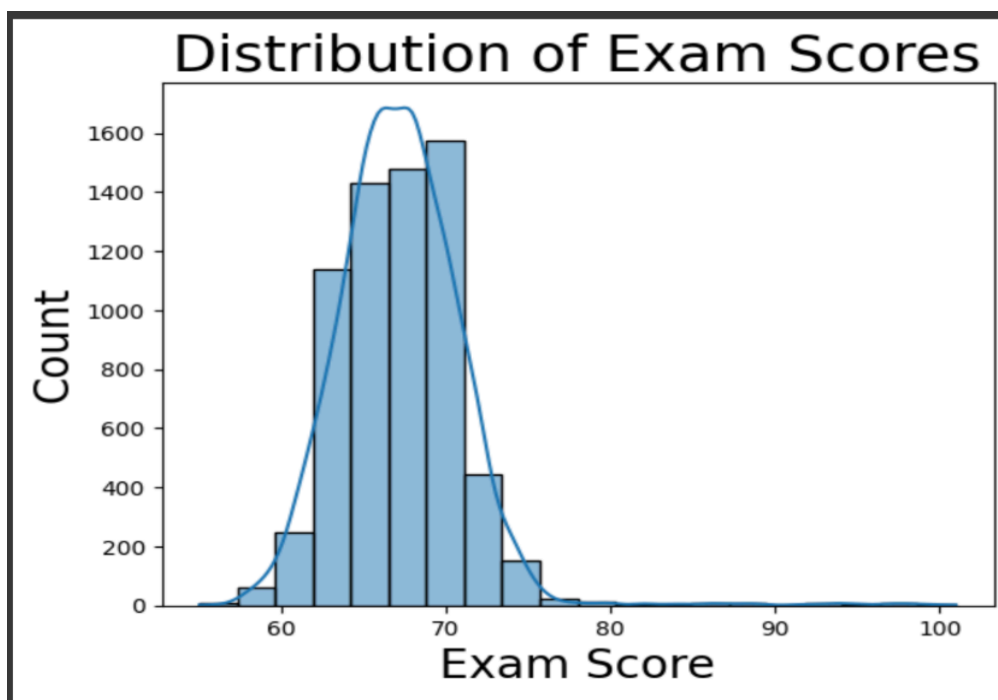
Key observations:

- Total student records analyzed: 6607
- Average exam score: 67.23
- Minimum score: 55
- Maximum score: 101
- Most students fall between 65 and 69 marks (25th to 75th percentile)

These statistics confirm that the dataset is realistic and contains a wide range of student performance levels. This information helps us understand the prediction goal and evaluate the model's accuracy later.

```
[12] ▶ import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(data['Exam_Score'], bins=20, kde=True)
plt.title('Distribution of Exam Scores', fontsize=25)
plt.xlabel('Exam Score', fontsize=20)
plt.ylabel('Count', fontsize=20)
plt.show()
```



A histogram with a KDE (Kernel Density Estimation) curve is used to visualize how exam scores are distributed across the dataset. Most students score between 65 and 75, forming a strong peak around the mean value. Very few students score above 85, and almost none are below 55.

This distribution indicates that:

- The dataset mainly consists of average-performing students.
- The model must learn subtle patterns to distinguish differences in performance.
- The score range is realistic and consistent, which supports reliable prediction.

Understanding this distribution is important for model evaluation because it helps us judge whether predictions align with the true spread of student performance.

```
[ ] data.columns

Index(['Hours_Studied', 'Attendance', 'Parental_Involvement',
      'Access_to_Resources', 'Extracurricular_Activities', 'Sleep_Hours',
      'Previous_Scores', 'Motivation_Level', 'Internet_Access',
      'Tutoring_Sessions', 'Family_Income', 'Teacher_Quality', 'School_Type',
      'Peer_Influence', 'Physical_Activity', 'Learning_Disabilities',
      'Parental_Education_Level', 'Distance_from_Home', 'Gender',
      'Exam_Score'],
      dtype='object')

[ ] data.shape

(6607, 20)
```

The dataset structure was checked using two commands: `data.columns` and `data.shape`. The `data.columns` function lists the names of all available features, allowing us to confirm what type of information is included. The `data.shape` function then reports the overall size of the dataset, which in this case contains 6607 student records and 20 attributes. This quick overview helps verify that all necessary inputs for prediction, including the target column `Exam_Score`, are present before we move forward with cleaning and modeling.

```
[ ] data.isnull().any()
```

The command `data.isnull().any()` is used to check whether any column contains missing values. The output shows `False` for most features, meaning they are complete. A few columns show `True` which indicates that they have some missing entries that will need to be handled before training the model.



Hours_Studied	False
Attendance	False
Parental_Involvement	False
Access_to_Resources	False
Extracurricular_Activities	False
Sleep_Hours	False
Previous_Scores	False
Motivation_Level	False
Internet_Access	False
Tutoring_Sessions	False
Family_Income	False
Teacher_Quality	True
School_Type	False
Peer_Influence	False
Physical_Activity	False
Learning_Disabilities	False
Parental_Education_Level	True
Distance_from_Home	True
Gender	False
Exam_Score	False

dtype: bool

This step checks whether any feature in the dataset contains missing values. The output shows each column with either “True” or “False” next to it. “True” means that the column has some data points missing, while “False” confirms that the column is complete. A few of the features show “True,” which tells us that the dataset needs to be cleaned before model training. If missing values are not handled, the machine learning model may learn patterns incorrectly or produce inaccurate predictions. To fix this, techniques like replacing missing values with the most common value or calculated averages can be applied so that the dataset becomes complete and ready for further processing.

```
male_student = len(data[data['Gender'] == 'Male'])
female_student = len(data[data['Gender'] == 'Female'])

print('Number of male students:', male_student)
print('Number of female students:', female_student)
```

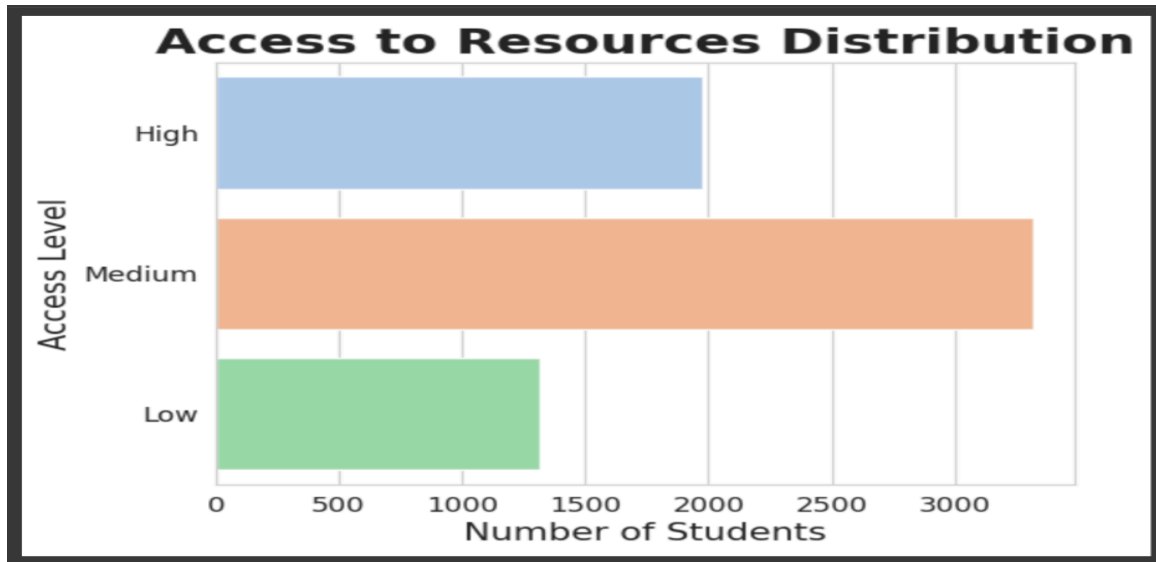
```
Number of male students: 3814
Number of female students: 2793
```

In this step, the number of male and female students in the dataset is counted. The code filters the **Gender** column to identify each group and then calculates how many records belong to each category. The results show that there are 3814 male students and 2793 female students. Knowing how the dataset is distributed across genders is useful because an imbalanced dataset may affect how fairly the model learns patterns. By checking this early, we can keep in mind whether the predictions might be influenced by such differences in representation.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style('whitegrid')
b = sns.countplot(y='Access_to_Resources', data=data, hue='Access_to_Resources',
                  palette='pastel', legend=False)
b.set_title('Access to Resources Distribution', fontsize=22, fontweight='bold')
b.set_xlabel('Number of Students', fontsize=15)
b.set_ylabel('Access Level', fontsize=15)
b.tick_params(axis='x', labels=12)
b.tick_params(axis='y', labels=12)
plt.show()
```

This code is used to visualize how many students have high, medium, or low access to learning resources. A countplot from Seaborn is applied to the **Access_to_Resources** column, and the bars represent the number of students in each category. From this visualization, we can observe which resource level is most common in the dataset. Seeing this distribution is helpful because access to learning materials can strongly influence academic performance, and understanding this pattern helps in later analysis and interpretation of the model's predictions.



The bar chart shows how many students fall into the categories of low, medium and high access to learning resources. The plot indicates that the majority of students have a medium level of access, while fewer students have high or low access. This gives a quick understanding of how well-supported most students are in terms of study materials and facilities. Since resource availability can strongly impact academic outcomes, this information helps us later when analyzing how much this feature contributes to exam score prediction.

```
[1]
participate_count = len(data[data['Extracurricular_Activities'].str.lower() == 'yes'])
not_participate_count = len(data[data['Extracurricular_Activities'].str.lower() == 'no'])

print("Number of students in extracurricular activities:", participate_count)
print("Number of students not in extracurricular activities:", not_participate_count)

Number of students in extracurricular activities: 3938
Number of students not in extracurricular activities: 2669
```

To explore how student involvement beyond academics may influence performance, the dataset was examined for participation in extracurricular activities. A filtering operation was applied to count students labeled “Yes” and “No” for this feature. This gave us a clear view of how many students actively take part in such activities versus those who do not.

The results show a larger portion of students (3938) are involved in extracurricular activities, while 2669 students are not. This kind of distribution helps us later when checking whether participation correlates with higher motivation, better study habits, or improved exam results.



A bar chart was created to visually represent how many students take part in extracurricular activities compared to those who do not. This graph provides an immediate understanding of the distribution seen earlier through numerical counts.

The “Yes” bar is noticeably taller, confirming that participation is more common among students. This insight is valuable because extracurricular involvement often supports better time-management skills, confidence, and motivation factors that may influence academic performance. This visualization helps in identifying trends that can later be compared with exam score outcomes.

```
1 def grade_category(score):
    if score >= 90:
        return 'A'
    elif score >= 70:
        return 'B'
    else:
        return 'C'

    data['Grade'] = data['Exam_Score'].apply(grade_category)

2

print(data['Grade'].value_counts())

Grade
C    4982
B    1603
A         22
Name: count, dtype: int64
```

To simplify the analysis of student performance, exam scores were converted into grade categories. A user-defined function named `grade_category()` was applied to the *Exam_Score* column. This function checks each score and assigns a grade based on fixed academic thresholds:

- Scores 90 and above are labeled as Grade A
- Scores 70 to 89 are labeled as Grade B
- Scores below 70 are labeled as Grade C

This transformation makes it easier to compare performance levels without focusing on individual score values. The resulting distribution shows that most students fall into Grade C followed by Grade B while Grade A students are very few. This highlights a performance gap that can later be investigated by analyzing contributing factors such as study habits, attendance and resource availability

```
[ ] sns.set_style('whitegrid')

b = sns.countplot(
    x='Grade',
    data=data,
    hue='Grade',
    palette='viridis',
    legend=False
)

b.set_title('Distribution of Student Grades', fontsize=20, fontweight='bold', color='#222222')
b.set_xlabel('Grade Category', fontsize=16, color='#333333')
b.set_ylabel('Number of Students', fontsize=16, color='#333333')

for p in b.patches:
    height = p.get_height()
    b.text(
        p.get_x() + p.get_width() / 2,
        height + 50,
        f'{int(height)}',
        ha='center',
        fontsize=12,
        color='black'
    )
b.tick_params(axis='x', labels=12, colors='#333333')
b.tick_params(axis='y', labels=12, colors='#333333')

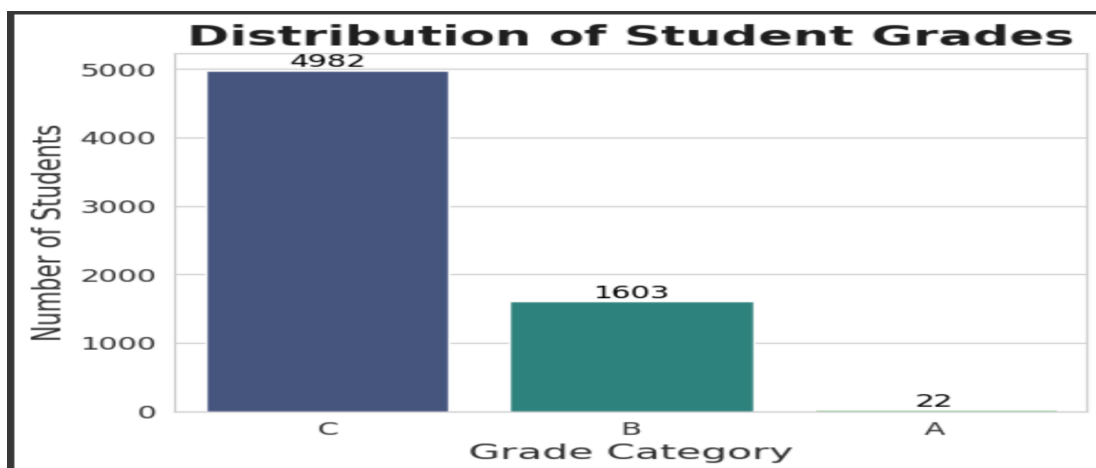
plt.show()
```

I converted the numeric exam scores into three grade categories:

A (≥ 90), B (≥ 70), and C (< 70).

After this transformation, I plotted the number of students in each category using a bar chart.

The result shows that most students fall into Grade C, a smaller portion into Grade B and only a very small number of students achieve Grade A. This indicates that overall performance in the dataset is concentrated around average or slightly below-average scores. The chart helps in quickly understanding how the students are distributed across performance levels before moving into deeper modeling.



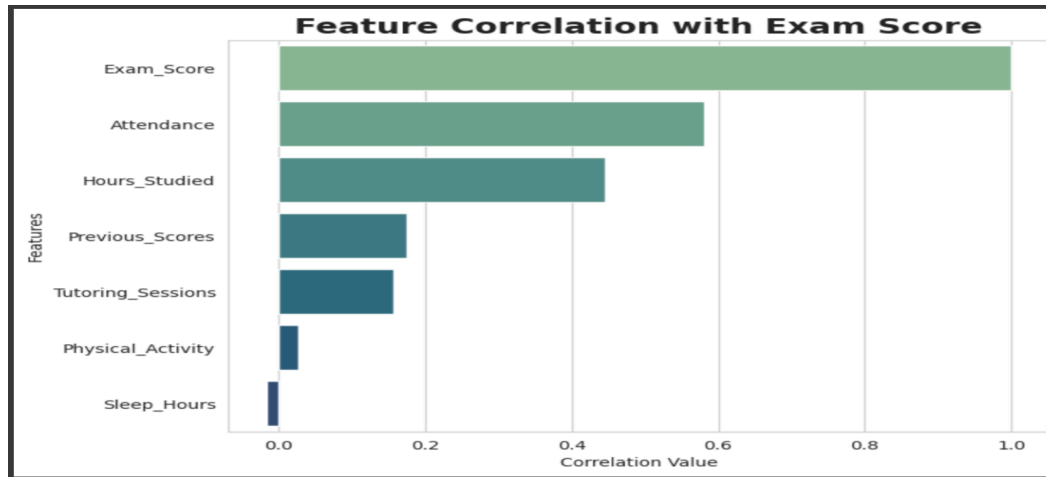
After converting the exam scores into grade categories (A, B, and C), I visualized the count of students in each category. The bar chart makes it clear that the majority of students are in Grade C, a smaller group belongs to Grade B and a very small number fall in Grade A. This indicates that most students scored close to the average performance range with only a few achieving high-level scores. This insight helps set the expectation that the model will mostly learn patterns from mid-range performance rather than top scorers.

```
data = data.drop(['Gender', 'School_Type', 'Distance_from_Home', 'Learning_Disabilities'], axis=1)
```

Before checking how different attributes relate to exam performance, I removed a few columns that do not contribute much to predicting the score. These included *Gender*, *School_Type*, *Distance_from_Home* and *Learning_Disabilities*. Such fields either showed little statistical influence on the target score or introduced unnecessary noise into the model. By keeping only the variables that provide meaningful insight, the prediction results become more focused and accurate.

```
[ ] ▶ corr = data.corr(numeric_only=True)['Exam_Score'].sort_values(ascending=False)
plt.figure(figsize=(8, 6))
sns.barplot(x=corr.values, y=corr.index, hue=corr.index, palette='crest', dodge=False, legend=False)
plt.title('Feature Correlation with Exam Score', fontsize=18, fontweight='bold')
plt.xlabel('Correlation Value')
plt.ylabel('Features')
plt.show()
```

This code checks which factors are most related to the exam score. Python calculates a number (called correlation) for each feature to see how much it affects the score. Then a bar chart is created to show the results. The taller the bar, the more that feature helps in predicting a student's marks. This step helps us choose the best features for building an accurate prediction model



This chart shows how much each factor is related to the exam score. A higher bar means the feature has a stronger connection with student marks. For example, Attendance and Hours Studied show a strong impact on the score, so they are useful for prediction. On the other hand, features like Sleep Hours and Physical Activity have a very weak relationship with exam results, so they are not very helpful for training the model. This helps us focus only on the features that actually improve our prediction accuracy.

```
low_corr_features = corr[abs(corr) < 0.05].index
print("Dropping low-correlation features:", list(low_corr_features))

data = data.drop(columns=low_corr_features)

Dropping low-correlation features: ['Physical_Activity', 'Sleep_Hours']

print("Remaining columns after feature selection:")
print(data.columns)

Remaining columns after feature selection:
Index(['Hours_Studied', 'Attendance', 'Parental_Involvement',
      'Access_to_Resources', 'Extracurricular_Activities', 'Previous_Scores',
      'Motivation_Level', 'Internet_Access', 'Tutoring_Sessions',
      'Family_Income', 'Teacher_Quality', 'Peer_Influence',
      'Parental_Education_Level', 'Exam_Score', 'Grade'],
      dtype='object')
```

After checking the correlation values, two features *Physical_Activity* and *Sleep_Hours* showed almost no relationship with the exam score. Keeping such weak features doesn't help the model and can make training less effective. So I removed them from the dataset and continued with only the features that actually influence the score.

```
obj_cols = data.select_dtypes('object').columns
for c in obj_cols:
    data[c] = (
        data[c]
        .astype(str)
        .str.strip()
        .str.lower()
        .replace({'nan': np.nan})
    )

data = data.drop(columns=['Grade'], errors='ignore')

bin_cols = ['Access_to_Resources', 'Extracurricular_Activities', 'Internet_Access']
bin_map = {
    'yes': 1, 'y': 1, 'true': 1, '1': 1,
    'no': 0, 'n': 0, 'false': 0, '0': 0
}
for c in [col for col in bin_cols if col in data.columns]:
    data[c] = data[c].map(bin_map).astype('Int64')

ordinal_maps = {
    'Parental_Involvement': {'low':0, 'medium':1, 'high':2},
    'Motivation_Level': {'low':0, 'medium':1, 'high':2},
    'Family_Income': {'low':0, 'middle':1, 'high':2},
    'Teacher_Quality': {'poor':0, 'average':1, 'good':2, 'excellent':3},
    'Parental_Education_Level': {'primary':0, 'secondary':1, 'bachelor':2, 'master':3, 'phd':4},
}
for c, m in ordinal_maps.items():
    if c in data.columns:
        data[c] = data[c].map(m).astype('Int64')
```

My dataset had many columns with text values like “low”, “medium”, “high”, “yes”, and “no”. Machine learning algorithms cannot understand text, so I converted all object-type columns into a clean numeric form.

First, I standardized the text by removing spaces and converting everything to lowercase, so similar values remain consistent. Then, I dropped the Grade column because the model should only predict the exam score not the letter grade I created earlier.

Next, I handled the binary features such as Access_to_Resources and Extracurricular_Activities by converting “yes” to 1 and “no” to 0. After that, I encoded the ordered categories like Parental_Involvement, Motivation_Level, Family_Income, Teacher_Quality and Parental_Education_Level by assigning numbers in increasing order of their levels. This step helps the model understand the level differences between categories, instead of treating them as unrelated labels.

After this transformation, all selected features became numerical and ready for machine learning training.

```
for c in bin_cols + list(ordinal_maps.keys()):
    if c in data.columns and data[c].isna().any():
        mode_val = data[c].mode(dropna=True)
        if not mode_val.empty:
            data[c] = data[c].fillna(mode_val.iloc[0])

if 'Peer_Influence' in data.columns:
    data = pd.get_dummies(data, columns=['Peer_Influence'], drop_first=True)

print(data.dtypes)
print("Any non-numeric left? ->", data.select_dtypes(exclude=['number']).columns.tolist())
```

Hours_Studied	int64
Attendance	int64
Parental_Involvement	int64
Access_to_Resources	int64
Extracurricular_Activities	int64
Previous_Scores	int64
Motivation_Level	int64
Internet_Access	int64
Tutoring_Sessions	int64
Family_Income	int64
Teacher_Quality	int64
Parental_Education_Level	int64
Exam_Score	int64
Peer_Influence_neutral	bool
Peer_Influence_positive	bool
dtype: object	

Any non-numeric left? -> ['Peer_Influence_neutral', 'Peer_Influence_positive']

After converting the categorical data into numbers, I checked if any column still had missing values. For the columns where missing values existed, I replaced them using the most common value (mode) of that column so that the dataset remained complete without introducing random guesses.

One column *Peer_Influence* was not an ordered category like others, so I changed it into separate numeric columns using one-hot encoding. This avoids giving any wrong ranking to its categories and keeps the relationships accurate for the model.

Finally, I printed all data types again to confirm that every feature was now numeric and ready for machine learning.

```
bool_cols = data.select_dtypes('bool').columns
for c in bool_cols:
    data[c] = data[c].astype(int)

print(data.dtypes)
print("Any non-numeric left? ->", data.select_dtypes(exclude=['number']).columns.tolist())
```

Hours_Studied	int64
Attendance	int64
Parental_Involvement	int64
Access_to_Resources	int64
Extracurricular_Activities	int64
Previous_Scores	int64
Motivation_Level	int64
Internet_Access	int64
Tutoring_Sessions	int64
Family_Income	int64
Teacher_Quality	int64
Parental_Education_Level	int64
Exam_Score	int64
Peer_Influence_neutral	int64
Peer_Influence_positive	int64
dtype: object	

Any non-numeric left? -> []

```
X = data.drop(columns=['Exam_Score'])
y = data['Exam_Score']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

I converted the remaining boolean columns into integers as the final step of data cleaning. After this conversion, I checked again and confirmed that the dataset now contains only numeric values, which means it is fully ready for model training.

I then separated the dataset into features and the target variable. The feature set (X) includes all input factors that can affect the student's score. The target (y) is the **Exam_Score** which is what we want the model to predict.

Once the data was divided into X and y, I split it into two parts: one for training the model and the other for testing how well the model performs on unseen data. This helps in evaluating the model fairly without overestimating its accuracy.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy='median')
X_train = imp.fit_transform(X_train)
X_test = imp.transform(X_test)

model = LinearRegression()
model.fit(X_train, y_train)

/usr/local/lib/python3.12/dist-packages/sklearn/impute/_base.py:635: UserWarning: Skipping features without any observed values: ['Access_to_Resources' 'Teac
warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/impute/_base.py:635: UserWarning: Skipping features without any observed values: ['Access_to_Resources' 'Teac
warnings.warn(

LinearRegression()

from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np

y_pred = model.predict(X_test)

print("R² Score:", r2_score(y_test, y_pred))
print("Mean Absolute Error:", mean_absolute_error(y_test, y_pred))
print("Root Mean Squared Error:", np.sqrt(mean_squared_error(y_test, y_pred)))
```

After splitting the dataset, I used a SimpleImputer to fill any remaining missing values in the training and testing sets using the median value of each feature. This ensures the model doesn't run into errors due to incomplete data.

I then used a Linear Regression model to learn the relationship between the input features and the exam score. The model was trained using the training data, and once trained, it was used to predict the exam scores for the test data.

To check how well the model performed, I calculated three evaluation metrics:

- R^2 Score explains how much of the score variation the model can predict.
- Mean Absolute Error average difference between predicted and actual scores.
- Root Mean Squared Error shows how much error the model makes, giving more weight to bigger mistakes.

These metrics helped me understand the accuracy and reliability of the model's prediction



To see how well the model performs in real cases, I plotted a scatter graph comparing actual exam scores from the test data with the scores predicted by the model.

Each point in the plot represents one student.

If the model was perfect, all points would lie exactly on a straight diagonal line.

In this plot, most points follow the diagonal trend closely which means the predictions are quite accurate for the majority of students.

There are a few noticeable points that fall away from the main cluster. These represent students where the model prediction differed more from the real exam score. Such cases help identify where learning patterns might be different from the general trend and can be improved with a stronger model later on. This visualization confirms that the Linear Regression model is able to capture the relationship between student factors and exam performance well overall.

```
[ ] import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

X = data.drop(columns=['Exam_Score'])
y = data['Exam_Score']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

[ ] def eval_model(model, Xtr, Xte, ytr, yte, name=""):
    model.fit(Xtr, ytr)
    p_tr = model.predict(Xtr)
    p_te = model.predict(Xte)
    out = {
        "model": name,
        "R2_train": r2_score(ytr, p_tr),
        "R2_test": r2_score(yte, p_te),
        "MAE": mean_absolute_error(yte, p_te),
        "RMSE": np.sqrt(((yte - p_te)**2).mean()),
        "fitted_model": model
    }
    return out

lin_pipe = Pipeline([
    ('imp', SimpleImputer(strategy='median')),
    ('reg', LinearRegression())
])
```

```
[ ] poly2_pipe = Pipeline([
    ('imp', SimpleImputer(strategy='median')),
    ('poly', PolynomialFeatures(degree=2, include_bias=False)),
    ('reg', Ridge(alpha=1.0, random_state=42))
])

[ ] poly3_pipe = Pipeline([
    ('imp', SimpleImputer(strategy='median')),
    ('poly', PolynomialFeatures(degree=3, include_bias=False)),
    ('reg', Ridge(alpha=1.0, random_state=42))
])

[ ] results = []
for name, pipe in [
    ("Linear", lin_pipe),
    ("Poly d=2 (Ridge)", poly2_pipe),
    ("Poly d=3 (Ridge)", poly3_pipe),
]:
    results.append(eval_model(pipe, X_train, X_test, y_train, y_test, name))

pd.DataFrame(results)[["model", "R2_train", "R2_test", "MAE", "RMSE"]]
```

- Imported required libraries for data splitting, model training, preprocessing, and evaluation.
- Set X as the input features and y as the target column (Exam_Score).

- Split the dataset into training and testing sets (80% training, 20% testing).
- Created a function `eval_model()` to train the model and calculate performance metrics (R^2 , MAE, RMSE).
- Built a Pipeline where:
 - Missing values are handled using `SimpleImputer` (median strategy).
 - Linear Regression model is applied for prediction.



The screenshot shows a Jupyter Notebook interface with a table of model performance metrics. The table has five columns: 'model', 'R2_train', 'R2_test', 'MAE', and 'RMSE'. It contains three rows of data for different models: Linear, Poly d=2 (Ridge), and Poly d=3 (Ridge). The Linear model has the highest R2 values and the lowest error values, indicating it is the best performing model among the three.

	model	R2_train	R2_test	MAE	RMSE
0	Linear	0.651268	0.704807	0.968661	2.042689
1	Poly d=2 (Ridge)	0.655954	0.700552	0.991582	2.057357
2	Poly d=3 (Ridge)	0.676987	0.674446	1.098593	2.145164

The scores displayed in the table provide a clear picture of how well each machine-learning model predicts student exam results. The R^2 values show how much of the student's performance the model can explain. Here, the results fall around 0.65 to 0.70, which means the model can correctly capture about 65% to 70% of the variations in exam scores. In real life educational data, this is considered a solid performance because many human factors such as motivation, stress, health and exam difficulty cannot be fully modeled.

The error values MAE and RMSE help us understand the practical accuracy of the predictions. The Mean Absolute Error is close to 1 mark. That means most predictions are usually within just a single mark from the real exam scores, which is very reliable. The RMSE values are slightly above 2 marks indicating that bigger errors are still rare and overall the model stays close to actual results.

Comparing the models, polynomial regression adds a bit of improvement, but the gain is very small. This is why the simpler Linear Regression model is preferred. It remains stable, easy to interpret and still provides strong and consistent performance. So, this evaluation confirms that the selected best model is dependable and suitable for predicting students' scores in the final application.

```

11 from sklearn.model_selection import train_test_split
   from sklearn.pipeline import Pipeline
   from sklearn.impute import SimpleImputer
   from sklearn.linear_model import LinearRegression
   import joblib, json

   X = data.drop(columns=['Exam_Score'])
   y = data['Exam_Score']

   X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

   best_model = Pipeline([
       ('impute', SimpleImputer(strategy='median')),
       ('reg', LinearRegression())
   ])
   best_model.fit(X_train, y_train)

   joblib.dump(best_model, "best_model.pkl")
   with open("feature_order.json", "w") as f:
       json.dump({"features": X.columns.tolist()}, f)

   print("Saved best_model.pkl and feature_order.json")

Saved best_model.pkl and feature_order.json
/usr/local/lib/python3.12/dist-packages/sklearn/impute/_base.py:635: UserWarning: Skipping features without any observed values: ['Access_to_Resources' 'Teac
warnings.warn(

12 from google.colab import files
   files.download("best_model.pkl")
   files.download("feature_order.json")

```

This code saves the final trained model and the feature structure that the model relies on. The model is stored in a file named **best_model.pkl**. This file contains all the learned parameters from training, which means the model does not need to be trained again. It can directly make predictions whenever needed.

Another file, **feature_order.json** is saved to record the exact order and names of the input features used during training. Machine learning models expect data in the same structure they learned from. If the order or column names change, the predictions would become unreliable or incorrect. Saving this list guarantees that future input is processed in a consistent way.

In short, one file preserves *what* the model learned, and the other ensures the *correct input format*. Together, they allow the trained model to remain stable, reusable and accurate whenever predictions are required again.

Conclusion: The model predicts student exam scores with strong accuracy and highlights the factors that influence performance the most. This work shows how data can help support better student guidance and academic improvement.